CSA47 - Deep Learning

Assignment 2: CO2 - Deep Feed-Forward and Convolutional Network Design

Field	Details
Student name	A HARITHA
Register number	192321147
Department	Information Technology, SIMATS Engineering
Assignment	CO2 - Deep Feed-Forward and CNN Design (Smart Agriculture), 50 marks

ASSIGNMENT 2 - CO2: Deep Feed-Forward and Convolutional Network Design

Notation. For layer l with n_l neurons, the weight matrix $W^{(l)}$ has shape $n_l \times n_{l-1}$ and the bias $b^{(l)}$ has n_l entries. Forward pass: $a^{(l)} = \text{phi}(W^{(l)} a^{(l-1)} + b^{(l)})$.

Task 1 - Deep Feed-Forward Network Architecture (5 marks)

(a) Architecture. Four inputs (rainfall, temperature, soil moisture, fertilizer quantity), hidden layers of 8 and 4 neurons, and one linear output for continuous yield.

Layer	Neurons	Activation	Purpose
Input	4	None	Receives four normalized measurements
Hidden 1	8	ReLU	First-level nonlinear combinations
Hidden 2	4	ReLU	Higher-level combinations
Output	1	Linear	Predicts crop yield

Architecture: **4 - 8 - 4 - 1**.

(b) Weight and bias dimensions (column-vector convention, input x of shape 4 x 1):

Parameter	Shape	Reason
W^1	8 x 4	Maps 4 inputs to 8 hidden units
b^1	8 x 1	One bias per first-hidden unit
W^2	4 x 8	Maps 8 activations to 4 units
b^2	4 x 1	One bias per second-hidden unit
W^3	1 x 4	Maps 4 activations to one output
b^3	1 x 1	Output bias

Total trainable parameters: (32 + 8) + (32 + 4) + (4 + 1) = **81**. With a batch of m examples stored as rows, the input is m x 4; either convention works if used consistently.

(c) Forward propagation.

```
h1 = ReLU(W1 x + b1) 8 x 1 h2 = ReLU(W2 h1 + b2) 4 x 1 y_hat = W3 h2 + b3 1 x 1
```

- Normalize the four inputs with training-set statistics.
- Multiply by W1, add b1 to get eight pre-activations, and apply ReLU to get h1.
- Pass h1 through W2 and b2, then ReLU, to get h2.
- Apply W3 and b3 with a linear output to get the predicted yield.

A linear output suits unconstrained regression. Train with MSE (or MAE if outliers matter), monitor validation loss, and evaluate on farms or seasons not used in training.

Task 2 - Forward Propagation and Activation Functions (2 marks)

Inputs x = (0.6, 0.8, 0.5), weights w = (0.4, 0.3, 0.2), bias b = 0.1.

```
(a) z = 0.4(0.6) + 0.3(0.8) + 0.2(0.5) + 0.1 = 0.24 + 0.24 + 0.10 + 0.10 = 0.68
(b) ReLU(z) = max(0, 0.68) = 0.68
```

The neuron is active, so ReLU passes its input unchanged. For a negative weighted sum ReLU would output 0, which is why ReLU units can become inactive for some inputs. The nonlinearity is what lets stacked layers represent more than one linear map.

Task 3 - Gradient-Based Learning and Loss Calculation (6 marks)

(a) Why a loss is needed. The loss turns the gap between prediction and target into a number that training can minimize. For continuous yield, mean squared error is a standard choice: it is differentiable and penalizes large errors strongly. MAE is more robust to outliers; Huber loss combines both behaviours.

$$MSE = (1/N) \sum_i (y_i - \hat{y}_i)^2$$

(b) Calculation. Actual y = 50, predicted $\hat{y}$ = 42:

$$L = (y - \hat{y})^2 = (50 - 42)^2 = 8^2 = 64$$

(c) Interpretation. The prediction is 8 yield units below the actual value. The loss of 64 is in squared yield units, so it is not a percentage. Its square root (8) is the absolute error in yield units. A prediction of 48 would give L = 4, meaning lower loss corresponds to a closer prediction, but this says nothing yet about unseen farms.

(d) Gradient-based updates. A forward pass gives $\hat{y}$ and L. Backpropagation computes dL/dtheta for every weight and bias. The optimizer then moves each parameter against its gradient:

$$\theta_{new} = \theta_{old} - \eta \cdot \frac{dL}{d\theta}$$

$$\frac{dL}{d\hat{y}} = -2 (y - \hat{y}) = -2 (50 - 42) = -16$$

The negative derivative means that increasing the prediction would reduce the loss. The chain rule propagates this signal back through each layer to every weight. For a mini-batch, the loss is averaged over the batch, giving an estimate of the full-data gradient, and the cycle repeats over many batches and epochs.

Task 4 - Gradient Descent and Weight Update (4 marks)

(a) w_old = 0.5, dL/dw = 0.8, eta = 0.1.

$$w_{new} = w_{old} - \eta \frac{dL}{dw} = 0.5 - 0.1(0.8) = 0.5 - 0.08 = 0.42$$

(b) Direction. The gradient points uphill, in the direction in which the loss increases fastest. Subtracting it moves the parameter downhill. This is a local, first-order rule, valid for small steps.

(c) Learning-rate effects. Two illustrations on $L(w) = w^2$ (gradient 2w):

- **Very small eta = 0.01:** from w = 0.5, the first step gives 0.5 - 0.01(1) = 0.49. Progress is stable but slow, so many epochs are needed.
- **Very large eta = 1.5:** from w = 0.5, the gradient is 1, so w becomes 0.5 - 1.5 = -1.0. The next gradient is -2, giving w = 2.0, then -4.0. The iterates oscillate with growing amplitude and diverge (possibly to NaN).

Remedies: use a learning-rate sweep or schedule, warm-up for large models, gradient clipping for unstable gradients, and tune the rate on validation performance, not training loss alone.

Task 5 - Back-Propagation Through Multiple Layers (3 marks)

(a) How backpropagation attributes error. After the forward pass, the output error signal delta = dL/dy_hat is computed. It is then passed backward: each layer multiplies the incoming error by the transpose of its weight matrix and by the activation derivative, producing the error signal for the layer below. A weight's gradient is its layer's error signal times the activation feeding into it, so weights with large gradients have the most influence on the error.

(b) Chain rule. A deep network is a composition of functions. For a loss through two hidden layers:

```
dL/dW1 = (dL/dy_hat)(dy_hat/dh2)(dh2/dz2)(dz2/dh1)(dh1/dz1)(dz1/dW1)
```

Each factor is a local derivative, and backpropagation reuses intermediate results so that the full chain is never recomputed.

(c) Gradients to updates. $W^{(l)}$ -←gt; $W^{(l)}$ - eta $dL/dW^{(l)}$ and $b^{(l)}$ -←gt; $b^{(l)}$ - eta $dL/db^{(l)}$. Backpropagation computes the gradients; gradient descent, momentum or Adam uses them to update parameters. They are related but distinct operations.

Task 6 - Activation Functions and Vanishing/Exploding Gradients (3 marks)

(a) Vanishing gradients. Gradients through many layers are products of weight matrices and activation derivatives. The sigmoid derivative sigma(z)(1 - sigma(z)) is at most 0.25. Across ten sigmoid layers the product of best-case derivatives is 0.25^{10} = about 9.5 x 10^{-7}, so early layers barely learn. Products of large factors instead cause exploding gradients.

(b) Comparison for hidden layers.

Activation	Range	Derivative	Strengths	Weaknesses
Sigmoid	(0, 1)	Max 0.25	Probability-like output	Saturates; not zero-centred; vanishing gradients
tanh	(-1, 1)	Max 1 (at 0)	Zero-centred	Still saturates; vanishing gradients in deep stacks
ReLU	[0, inf)	1 for z > 0, else 0	Cheap; no saturation for positive input	Dying units for negative inputs

(c) Effect on training. ReLU is the usual default for hidden layers because its positive-region derivative is 1. Leaky ReLU, GELU or SiLU reduce dead units. Sigmoid suits binary outputs and softmax suits mutually exclusive classes. Activation choice works best with matched initialization (He for ReLU, Xavier for tanh/sigmoid), normalization layers, residual connections and, if needed, gradient clipping.

Task 7 - Back-Propagation and Optimization Strategy (4 marks)

(a) Learning rate. The learning rate sets the step size. Too small: slow loss decrease. Too large: the iterates overshoot, oscillate or diverge. With mini-batches, each batch is a noisy sample, so the loss also fluctuates; a larger batch reduces this noise.

(b) One training iteration.

- Draw a mini-batch of inputs and targets.
- Run forward propagation to get predictions.
- Compute the loss, including any regularization term.
- Backpropagate with the chain rule to get gradients.
- Apply the optimizer update to weights and biases.
- Record training and validation metrics.

(c) Suggested changes.

Observed problem	Suggested changes
Loss decreases slowly	Check input and target scaling; raise the learning rate cautiously (learning-rate range test); use Adam or momentum; verify initialization and gradients
Loss falls quickly but fluctuates	Lower the learning rate; add a decay schedule; use momentum or tuned Adam; increase batch size if resources allow; inspect outliers and noisy labels; clip gradients

(d) Why monitor the loss curve. Training loss shows how well the optimizer fits the training data; validation loss shows whether the improvement transfers. Falling training loss with rising validation loss signals overfitting. Both high signals underfitting, optimization failure or weak features. Use early stopping on a chosen validation metric and keep the best checkpoint.

Task 8 - Regularization and Overfitting Control (4 marks)

(a) Diagnosis. Training accuracy of 99% against validation accuracy of 78% is a 21-point gap, which indicates **overfitting**: the network memorizes training examples rather than the crop-category pattern. Before changing the model, check for leakage (the same plant or field in both sets), duplicates, distribution shift, label quality and validation size.

(b) L1 and L2 regularization.

- **L1** adds lambda sum |w|. Its gradient is a constant push toward zero, so it drives some weights exactly to zero, giving sparsity and implicit feature selection.
- **L2** adds lambda sum w^2. Its gradient 2 lambda w shrinks weights in proportion to their size, discouraging large weights and spreading learning across features rather than zeroing them.

(c) Regularized loss (L2).

```
L_total = L_data + lambda sum over j of w_j^2
```

L_data is MSE for regression or cross-entropy for classification. lambda controls the strength: too small leaves overfitting unchecked; too large suppresses useful signal and causes underfitting. Biases are usually excluded. Some texts use (lambda/2) sum w^2, so the convention should be documented.

(d) Dropout. During training, each activation is zeroed with probability p. The network cannot rely on any single unit, so it learns redundant features; this is often interpreted as averaging an ensemble of sub-networks. Frameworks use inverted dropout (scaling kept activations by 1/(1 - p)) and disable dropout at inference. Early stopping, augmentation, weight decay and more representative data are complementary measures.

Task 9 - Convolution Operation and Feature Extraction (3 marks)

(a) Why convolution suits images. Visual signs such as spots, edges and texture are local patterns. A small filter scanned across the image reuses the same weights at every position (weight sharing), so it needs far fewer parameters than a dense layer connected to every pixel, and it detects a pattern wherever it appears.

(b) Worked example. Stride 1, no padding, cross-correlation (the operation used in deep-learning libraries):

```
Image I = [[1, 0, 2, 1], Filter K = [[ 1, 0], [0, 3, 1, 0], [ 0, -1]] [2, 1, 0, 1], [1, 0, 1, 2]]

Top-left window [[1,0],[0,3]]: 1(1) + 0(0) + 0(0) + 3(-1) = 1 - 3 = -2 Window at (0,2) [[2,1],[1,0]]: 2(1) + 1(0) + 1(0) + 0(-1) = 2 Window at (1,1) [[3,1],[1,0]]: 3(1) + 1(0) + 1(0) + 0(-1) = 3
```

Completing all nine windows gives the 3 x 3 feature map:

```
O = [[-2, -1, 2], [-1, 3, 0], [ 2, 0, -2]]
```

Interpretation. The filter responds to differences along the main diagonal: the largest positive response (3) sits where a bright pixel is followed diagonally by a dark one, and negative values mark the opposite change. A trained CNN learns such filters from data; this one is hand-chosen to show the arithmetic.

(c) Filters, stride and padding. Filters (kernels) are learnable weight patches; the number of filters sets output channels. Stride is the step size: larger strides shrink the map and cut computation but can skip small lesions. Padding adds a border (usually zeros) that preserves size and border information. For each spatial dimension:

```
Output = floor((N + 2P - F) / S) + 1 Example: N=4, F=2, P=0, S=1 -> floor(2/1)+1 = 3
(matches the 3 x 3 map) Example: N=32, F=3, P=1, S=2 -> floor(31/2)+1 = 16
```

Task 10 - CNN Architecture and Pooling (4 marks)

(a) Components.

Component	Function
Input image	Pixel tensor of height x width x channels (for example 224 x 224 x 3)
Convolution	Learns local filters and produces feature maps
ReLU	Adds nonlinearity so the network can model complex visual patterns
Pooling	Downsamples the map, reducing computation and adding small translation tolerance
Second convolution and ReLU	Combines simple features into more complex ones
Second pooling	Further reduces spatial size
Fully connected layer	Combines extracted features for the decision
Output (softmax)	Class probabilities for mutually exclusive classes

(b) Pooling. 2 x 2 max pooling with stride 2 takes the maximum of each non-overlapping 2 x 2 block:

```
Input = [[1, 2, 5, 3], [4, 0, 1, 2], [3, 1, 6, 0], [2, 4, 1, 8]] Blocks: max(1,2,4,0)=4
max(5,3,1,2)=5 max(3,1,2,4)=4 max(6,0,1,8)=8 Output = [[4, 5], [4, 8]] (4 x 4 -> 2 x 2)
```

Each pooling step halves height and width, so the next layer processes about a quarter of the positions. Pooling has no learned weights; average pooling uses block means instead.

(c) Filter size, stride and filter count. Take a 224 x 224 x 3 input:

```
Conv1: 32 filters, 3x3, stride 1, pad 1 -> 224 x 224 x 32 params = 3*3*3*32 + 32 = 896
ReLU, Pool 2x2 stride 2 -> 112 x 112 x 32 Conv2: 64 filters, 3x3, stride 1, pad 1 -> 112
x 112 x 64 params = 3*3*32*64 + 64 = 18,496 ReLU, Pool 2x2 stride 2 -> 56 x 56 x 64
```

Larger filters cover more context but cost more weights. A larger stride reduces resolution and computation. More filters give richer representations but increase memory and parameters. Stride 2 with padding 1 on the 224 input gives floor((224+2-3)/2)+1 = 112, halving the map in one convolution.

(d) Hierarchical features. Early layers learn edges, colour contrasts and textures. Middle layers combine them into veins, spots and lesion boundaries. Deeper layers, with larger receptive fields, represent whole disease-related structures. The hierarchy is learned, not hand-coded. Deeper is not always better: small or imbalanced datasets may favour transfer learning from a compact pretrained model.

Task 11 - Comparative Evaluation of Feed-Forward and CNN Models (4 marks)

(a) Model A for numerical data. Rainfall, temperature, soil moisture and fertilizer are structured numeric features with no spatial order. A multilayer perceptron takes the fixed-length vector, applies dense transformations and learns nonlinear interactions, such as how fertilizer effects depend on moisture. It is a natural baseline for tabular data.

(b) Model B for images. A leaf image has spatial structure. Convolution exploits local connectivity and weight sharing, so edges, lesions and textures are detected wherever they occur, with far fewer parameters than dense connections.

(c) Technical comparison.

Dimension	Feed-forward (MLP)	CNN
Input	Fixed-length numeric vector	Image tensor H x W x C
Feature extraction	Dense combinations; benefits from scaling	Learned local filters, hierarchical spatial features
Parameter organization	Dense weight matrices	Shared kernels plus dense or pooled head
Spatial inductive bias	None	Local neighbourhoods and translation equivariance
Compute	Modest for few features	Grows with resolution, channels and depth; GPU-friendly
Typical output	Continuous yield	Disease class probabilities

A worked contrast: a dense layer from a 224 x 224 x 3 image (150,528 values) to 1,000 units has about 150 million weights, whereas the first convolutional layer above has 896.

(d) Recommendation. Use an MLP regressor for crop-yield prediction and a CNN for leaf disease classification. Evaluate each with task-appropriate metrics (MAE/RMSE/R^2 for yield; accuracy, macro-F1 and recall for disease). Do not compare a regression loss directly with classification accuracy, because the objectives differ.

Task 12 - Design Challenge: Optimize and Redesign the Deep Learning System (8 marks)

Decision: redesign the system into task-specific branches: an MLP for the 500,000 numeric records and a CNN for the 100,000 leaf images. Retain the original 4-8-4-1 network as the baseline for comparison.

A. Impact analysis (2 marks).

- **(a) Larger data and images.** 500,000 records x 4 float32 features = 8,000,000 bytes, about 8 MB for the raw matrix alone. A 224 x 224 x 3 image stored as uint8 is 150,528 bytes, so 100,000 images need about 15.1 GB uncompressed, and about 60.2 GB if stored as float32. Pipelines should decode, resize, augment and batch images on demand rather than loading everything. Compute for the CNN scales with resolution, channels, depth and number of epochs, so the image branch dominates training time.
- **(b) Limits of a feed-forward network on images.** Flattening destroys neighbourhood structure; the first dense layer creates a huge parameter count (150M weights in the example above); there is no translation-oriented inductive bias, so the network needs more data and compute to learn what a CNN gets from weight sharing.

B. Evaluation of the existing design (2 marks).

- **(a) Interpreting the results.** High training and lower validation performance signals overfitting or distribution mismatch: the model may be memorizing records, exploiting farm- or season-specific signals, or benefiting from leakage. The long training time is a separate engineering issue, pointing to inefficient batching or unnecessary capacity. Check the split, duplicates, label quality and class balance before changing architecture.
- **(b) Is more depth and width the answer?** No. Extra layers and neurons add capacity, training time and overfitting risk. When validation already lags training, more capacity without better data, regularization or validation design is unlikely to help. The right intervention depends on whether the problem is overfitting, underfitting, optimization or distribution shift. For images, the structural fix is a CNN, not a deeper MLP.

C. Proposed architectures and training strategy (2 marks).

Element	Numeric branch (MLP)	Leaf-image branch (CNN)
Input	4 normalized features; extra variables only if reliably available	Resized to 224 x 224 x 3 (or 128 x 128 if hardware limits)
Architecture	4 - 32 - 16 - 1, ReLU hidden layers; compare with the 4 - 8 - 4 - 1 baseline	EfficientNet-B0 pretrained on ImageNet, transfer learning, then selective fine-tuning

Element	Numeric branch (MLP)	Leaf-image branch (CNN)
Head / output	One linear neuron	Global average pooling, dropout 0.3, dense softmax (or sigmoid for multi-label)
Loss	MSE; Huber if outliers matter	Categorical cross-entropy
Optimizer	Adam, learning rate 1e-3	SGD with momentum 0.9 (lr about 0.01 from scratch) or Adam at 1e-4 for fine-tuning
Regularization	L2 weight decay 1e-4; early stopping (patience 10); dropout only if validation supports it	Augmentation; dropout in head; weight decay; early stopping; batch normalization inside the backbone
Metrics	MAE, RMSE, R-squared; errors by farm and season	Accuracy, macro-F1, per-class recall, confusion matrix, ROC-AUC / PR-AUC

Optimizer comparison.

Optimizer	Summary	When to use
Batch gradient descent	Full dataset per update; stable but expensive	Small problems and reference runs
Mini-batch SGD	Batch of 32 to 256 per update; memory-efficient, noisy	Default scalable baseline
SGD with momentum	Accumulates a velocity from past gradients (typically 0.9)	CNN training with a tuned schedule
Adam	Per-parameter adaptive moments (beta1 0.9, beta2 0.999)	Fast practical start for the MLP and for fine-tuning
RMSProp	Scales updates by a moving average of squared gradients	Non-stationary or differently scaled gradients

Learning-rate schedule. Warm up linearly for about 3 epochs, then apply cosine decay to around 1e-6, or halve the rate (factor 0.5) when validation loss plateaus for 5 epochs, with a floor. Fine-tuning uses a lower rate for the backbone than for the new head. These are starting points, verified by monitoring validation loss and gradient stability.

Regularization and normalization. L2 weight decay; dropout in dense heads; batch normalization in the CNN backbone (it stabilizes training but does not replace input scaling); early stopping with checkpoint restoration; gradient clipping if exploding gradients are observed. Activations: ReLU in the MLP, with the built-in SiLU in EfficientNet and softmax at the output.

Label-preserving augmentation. Apply only to training images, mimicking plausible capture variation: small rotations (about +/-15 degrees), horizontal flips where biologically valid, small translations, mild zoom or crop (0.9 to 1.0), brightness and contrast changes of about 10-20%, and limited colour jitter. Avoid transformations that erase lesions or alter symptoms, such as heavy cutout over spots. Validation and test images use deterministic resizing and normalization. Split by plant, field or acquisition session before augmentation, so near-duplicate views cannot leak across splits.

Architecture choice for this budget.

Architecture	Characteristics	Advantages	Suitability here
LeNet	Small early CNN, few conv and pooling layers	Very low compute	Only for tiny, simple images
AlexNet	Deeper CNN with large early kernels and dense classifier	Historically important	Heavy dense layers; less efficient
ResNet (18/34/50)	Residual skip connections	Strong baseline; good gradient flow	Good fallback; ResNet-18/34 with transfer learning
EfficientNet (B0 first)	Compound scaling of depth, width and resolution	Strong accuracy per compute	Preferred starting backbone

The positions above are qualitative, not benchmark results. For 100,000 leaf images with limited compute, start from transfer learning with EfficientNet-B0 and compare with ResNet-18. If images are small or compute is very tight, a small custom CNN may suffice.

D. Evidence-based evaluation (2 marks).

(a) Experiment design.

- Create fixed, leakage-safe splits: training for fitting, validation for selection, and an untouched test set for reporting. Split numeric records by farm or season, and leaf images by plant, farm or source.
- Numeric branch: compare the original 4-8-4-1 network, the regularized 4-32-16-1 MLP, and a linear regression or gradient-boosted baseline.
- Image branch: compare a small custom CNN, transfer-learned EfficientNet-B0 and ResNet-18, plus an MLP on flattened images as a control that shows the value of convolution.
- Ablations: optimizer (Adam vs SGD with momentum), learning-rate schedule, augmentation, dropout, weight decay, batch normalization and input resolution.
- Repeat promising runs with 3 to 5 seeds and report mean and spread; compare configurations with paired tests or bootstrap intervals.
- Record loss curves, time per epoch, total training time, peak memory, inference latency and throughput.

Task	Metrics
Regression (yield)	MAE and RMSE in yield units; R-squared; residuals by yield range, farm and season
Classification (leaves)	Accuracy; precision and recall per class; macro-F1; confusion matrix; ROC-AUC / PR-AUC for rare classes
Operational	Training time per epoch and total; peak memory; inference latency; throughput

(b) Retain, modify or redesign? Redesign the overall system, retaining the original network as the baseline. High training performance with lower validation performance points to overfitting or distribution mismatch; the long training time calls for profiling and efficient batching; and images need a convolutional architecture. Adding depth alone is not justified.

Decision rules tied to the evidence:

- If the MLP lowers validation MAE/RMSE and holds up across farms and seasons, adopt it for yield prediction; if not, keep the 4-8-4-1 network.
- If validation loss diverges from training loss, add regularization, reduce capacity, improve data coverage or investigate leakage.
- If the pretrained CNN improves macro-F1 and disease-class recall within the compute budget, prefer it over training from scratch.
- If a smaller model performs similarly with lower latency and memory, deploy the smaller model.
- Do not deploy on training accuracy alone; verify performance on new farms, seasons, devices and lighting conditions.

Overall Conclusion

Architecture should match data type: a 4-8-4-1 feed-forward network for numeric yield, and a CNN for leaf images that exploits local spatial structure. The worked results are a weighted sum of 0.68 with ReLU output 0.68, a squared error of 64 for actual 50 and predicted 42, a gradient-descent update from 0.5 to 0.42, and 81 parameters in the 4-8-4-1 network. For the larger system, redesign into two branches with robust splits, training-only preprocessing, tuned optimizers, regularization, label-preserving augmentation and

evidence-based evaluation.

Final principle. High training accuracy is not evidence of a reliable agricultural model. Performance must generalize to new farms, seasons, devices and lighting conditions, and models should be monitored after deployment.